

Relatório Final — Seleção de Modelo de IA para Módulo de Análise Textual

Micro-capstone de pós-graduação — Benchmark comparativo de modelos de IA para produto SaaS B2B

Modelos avaliados: DeepSeek V4 Flash · Kimi K2.7 · Kimi K3

Status: benchmark executado na plataforma OpenCode (5 tickets × 3 modelos × 3 execuções em lote por modelo). Resultados preenchidos a partir dos logs reais de execução. O DeepSeek V4 Flash teve 1 execução descartada por instabilidade de rede e 3 válidas (1 na camada gratuita + 2 no plano pago); Kimi K2.7 e Kimi K3 tiveram 3 execuções válidas cada.

1. Capa

Título: Seleção de Modelo de IA para Módulo de Análise Textual em Produto SaaS B2B

Subtítulo: Benchmark comparativo entre DeepSeek V4 Flash, Kimi K2.7 e Kimi K3

Tarefa empresarial: Análise automatizada de tickets e feedbacks de clientes

Autor: Daniel Felipe Tomm

Data: 30/07/2026

2. Contexto empresarial

Uma startup B2B SaaS deseja adicionar inteligência artificial a um módulo de análise textual do produto, responsável por interpretar tickets de suporte e feedbacks de clientes. A decisão de qual modelo adotar afeta diretamente a margem do produto (custo por requisição multiplicado pelo volume), a experiência do usuário (latência e qualidade das análises), a escalabilidade da operação, a confiabilidade das classificações e o custo operacional de suporte.

A empresa precisa decidir qual modelo oferece o melhor equilíbrio entre desempenho técnico e viabilidade econômica, considerando também o risco empresarial de

automatizar decisões de atendimento.

3. Objetivo do benchmark

Comparar os modelos DeepSeek V4 Flash, Kimi K2.7 e Kimi K3 em qualidade, custo, velocidade, consistência, adequação ao formato e risco de erro, na tarefa real de negócio definida, de modo a fundamentar uma recomendação de adoção.

Hipótese de trabalho (neutra):

Modelos com maior custo podem apresentar melhor qualidade, mas o ganho pode não justificar o impacto econômico no produto SaaS.

4. Descrição da tarefa

Cada modelo recebe texto de ticket ou feedback de cliente e deve produzir, exclusivamente em JSON válido, uma análise contendo: resumo do problema, categoria do ticket, sentimento, urgência, risco de churn, ação recomendada, tom sugerido para a resposta e justificativa da classificação.

Esse caso de uso permite priorizar atendimentos, encaminhar tickets às equipes corretas, detectar clientes com risco de cancelamento, reduzir trabalho manual de triagem, melhorar o tempo de resposta e apoiar atendentes humanos.

A estrutura completa da saída esperada, as categorias permitidas e os riscos envolvidos estão detalhados no documento 01_descricao_tarefa_negocio.md.

5. Modelos avaliados

Modelo	Identificador técnico	Plataforma de execução
DeepSeek V4 Flash	deepseek-v4-flash (plano pago); deepseek-v4-flash-free (camada gratuita)	OpenCode (provedor opencode-go)
Kimi K2.7	kimi-k2.7-code	OpenCode (provedor opencode-go)
Kimi K3	kimi-k3	OpenCode (provedor opencode-go)

6. Metodologia

O benchmark seguiu o plano detalhado em 02_plano_benchmark.md, com um desvio registrado: em vez de uma execução individual por ticket, os 5 tickets foram enviados em um único prompt (lote) em cada execução. Foram realizadas 10 execuções no

total: 4 para o DeepSeek V4 Flash (1 descartada por instabilidade de rede, 1 válida na camada gratuita e 2 no plano pago) e 3 válidas para cada um dos modelos Kimi. As métricas de qualidade, consistência e latência do DeepSeek consideram as 3 rodadas válidas; as métricas de custo consideram apenas as rodadas do plano pago, que é a configuração viável para produção (a camada gratuita tem teto de 200 requisições/dia). A avaliação combinou métricas objetivas (custo e tokens reportados pela plataforma, latência medida pelos timestamps, validade do JSON, acurácia contra gabarito, consistência entre rodadas) e avaliação humana com rubrica de 1 a 5. A decisão final utilizou critérios eliminatórios e uma pontuação ponderada (0 a 100).

7. Dataset

Foram utilizados 5 tickets fictícios criados para a atividade, com gabarito de referência definido previamente:

- **Ticket 1 (comum):** erro 500 ao exportar relatório em PDF — problema técnico;
- **Ticket 2 (comum):** cobrança duplicada do plano anual;
- **Ticket 3 (comum):** dúvida de configuração de webhook — integração;
- **Ticket 4 (ambíguo):** reclamação de usabilidade com menção a "avaliar outras opções" na renovação;
- **Ticket 5 (difícil/crítico):** indisponibilidade de 3 horas no fechamento, com ameaça explícita de cancelamento.

8. Condições de comparação

Parâmetro	Condição definida	Condição observada
Tickets	Mesmos 5 tickets para todos os modelos	Atendido
Prompt	Prompt padrão idêntico	Atendido
Formato de saída	JSON conforme especificação	Atendido no prompt; DeepSeek cumpriu em 1 de 3 rodadas; Kimi K2.7 em 2 de 3 (bloco de código nas demais)
Ordem dos testes	Mesma ordem (tickets 1 a 5)	Atendido
Limite de resposta	Igual entre modelos, quando possível	Não controlado (limitação)
Temperatura	Zero ou menor valor disponível	Não controlado no OpenCode (limitação)
Repetições	1 por ticket por modelo	Desvio: execuções em lote; 3 rodadas válidas por modelo (DeepSeek: 1 gratuita + 2 pagas, após 1 descartada)

Parâmetro	Condição definida	Condição observada
Rede	Condições semelhantes	Parcialmente atendido: instabilidade de rede na 1ª execução do DeepSeek
Medição de latência	Mesmo critério em todas as execuções	Atendido (timestamps created/completed da plataforma)

9. Métricas

- **Custo por requisição:** custo reportado pela plataforma para cada execução (lote de 5 tickets), reconciliado com os preços unitários por milhão de tokens; média das rodadas (para o DeepSeek, apenas as rodadas pagas);
- **Latência:** diferença entre os timestamps de criação e conclusão; média, mediana e máximo (com 3 rodadas, o máximo aproxima o p95);
- **Qualidade:** acurácia de categoria, sentimento, urgência e risco de churn contra o gabarito, por rodada e em média; qualidade média da avaliação humana (rubrica 1 a 5);
- **Acurácia combinada de classificação:** média simples das quatro acurácias por rodada, usada no critério eliminatório de 80%;
- **Consistência:** percentual de tickets com classificação idêntica (categoria, sentimento, urgência e risco de churn) em todas as rodadas válidas do modelo;
- **Adequação ao formato e tom:** validade do JSON no formato exigido (sem texto ou blocos de código adicionais), presença dos campos, uso dos valores permitidos e cobertura dos 5 tickets;
- **Risco percebido de erro:** escala de 1 (muito baixo) a 5 (crítico);
- **Erros críticos:** tickets sem resposta ou com erro que comprometeria a automação.

10. Resultados

Métrica	DeepSeek V4 Flash	Kimi K2.7	Kimi K3
Rodadas válidas	3 (1 gratuita + 2 pagas)	3	3
Tokens médios (entrada/saída)	3.076 / 903*	5.177 / 2.024	5.725 / 1.302
Custo médio por execução (USD)	0,00109884 (pagas)	0,01351936	0,07587480
Latência média (ms)	17.119	13.212	110.128
JSON válido no formato exigido (%)	33,3 (1/3)	66,7 (2/3)	100 (3/3)
Consistência entre rodadas (%)	40	60	60

Métrica	DeepSeek V4 Flash	Kimi K2.7	Kimi K3
Qualidade média (1 a 5)	5,0	4,0	4,5
Risco percebido (1 a 5)	2	2	2

* As rodadas do DeepSeek consumiram entre 680 e 954 tokens de reasoning, cobrados como tokens de saída; duas delas reutilizaram cache de sessão (9.600 e 9.088 tokens), reduzindo o custo. O modelo respondeu os 5 tickets em todas as rodadas válidas: 2 vezes em array JSON envolvido em bloco de código markdown e 1 vez em JSON puro, porém com envelope de objeto por ticket (ticket_1 a ticket_5) em vez de array — o desvio de formato, antes consistente, mostrou-se mitigável na última rodada, mas a estrutura do envelope variou. O Kimi K2.7 retornou JSON puro em 2 de 3 rodadas; o Kimi K3, nas 3. O Kimi K3 consumiu entre 1.214 e 3.422 tokens de reasoning por rodada, o que explica suas latências elevadas (81 a 148 s).

11. Comparação de qualidade

Métrica	DeepSeek V4 Flash	Kimi K2.7	Kimi K3
Acurácia de categoria (%)	100	100	100
Acurácia de sentimento (%)	100	100	100
Acurácia de urgência (%) — média	86,7	40	60
Acurácia de risco de churn (%) — média	86,7	66,7	73,3
Acurácia combinada por rodada (%)	100 / 85 / 95	75 / 80 / 70	90 / 80 / 80
Acurácia combinada média (%)	93,3	75	83,3
Qualidade média da avaliação humana (1 a 5)	5,0	4,0	4,5

Análise: os três modelos acertaram categoria e sentimento de todos os tickets em todas as rodadas — a diferenciação está na calibragem de urgência e churn. O DeepSeek V4 Flash obteve a melhor qualidade média do benchmark (93,3% combinado), com duas rodadas em 100% e 95% e uma em 85%; suas divergências concentraram-se na urgência dos tickets 1 e 2 (rodada paga intermediária) e no churn do ticket 4 (rodadas pagas). O Kimi K3 manteve padrão estável de divergência (urgência alta nos tickets 1 e 2 nas 3 rodadas; churn superestimado nos tickets 2 e 4 nas rodadas 2 e 3), com média de 83,3%. O Kimi K2.7 foi o mais propenso a superestimar urgência (acurácia média de 40% nesse campo), com média de 75%. Com 3 rodadas e 5 tickets, o DeepSeek se consolida como líder em qualidade, embora a amostra ainda não permita conclusão estatística robusta.

12. Comparação de custo

Preços por milhão de tokens vigentes na data do teste (30/07/2026), conforme tabela da plataforma:

Métrica	DeepSeek V4 Flash	Kimi K2.7	Kimi K3
Preço de entrada (USD/milhão)	0,14	0,95	3,00
Preço de saída (USD/milhão)	0,28	4,00	15,00
Leitura de cache (USD/milhão)	0,0028	0,19	0,30
Limite da camada gratuita	200 requisições/dia*	Não se aplica	Não se aplica
Custo médio por execução — lote de 5 tickets (USD)	0,00109884	0,01351936	0,07587480
Custo médio por ticket (USD)	0,00021977	0,00270387	0,01517496
Custo em 10 mil execuções (USD)	10,99	135,19	758,75
Custo em 100 mil execuções (USD)	109,88	1.351,94	7.587,48
Custo em 1 milhão de execuções (USD)	1.098,84	13.519,36	75.874,80

* Aplica-se apenas à camada gratuita (deepseek-v4-flash-free); o plano pago não possui esse teto. A plataforma também exibe uma coluna "Usage" de US\$ 60 para o modelo, cujo significado não foi documentado neste teste.

Os custos reportados pela plataforma reconciliam com os preços unitários em todas as rodadas. Nos três modelos, os tokens de reasoning são cobrados como tokens de saída, e rodadas com leitura de cache saíram mais baratas (ex.: 2ª rodada paga do DeepSeek, com 9.088 tokens de cache, custou US\$ 0,00046; 3ª rodada do Kimi K3, com 8.448 tokens de cache, custou US\$ 0,0422 contra US\$ 0,0971 da 2ª).

Análise: o DeepSeek V4 Flash no plano pago é, de longe, a opção mais barata: US\$ 0,00022 por ticket em média — cerca de 12× mais barato que o Kimi K2.7 e 69× mais barato que o Kimi K3. Em 1 milhão de execuções em lote (5 milhões de tickets), custaria ~US\$ 1,1 mil, contra ~US\$ 13,5 mil do K2.7 e ~US\$ 75,9 mil do K3. A camada gratuita (custo zero) permanece utilizável para volumes de até 200 requisições/dia (~1 mil tickets/dia em lotes de 5). As projeções consideram a execução em lote de 5 tickets; em produção, com uma requisição por ticket, os valores por ticket devem ser usados como base. Os preços e limites são os vigentes na data do teste e podem mudar.

13. Comparação de latência

Métrica	DeepSeek V4 Flash	Kimi K2.7	Kimi K3
Latências por rodada (ms)	19.492 / 13.537 / 18.328	13.051 / 11.487 / 15.099	101.318 / 147.993 / 81.074
Latência média (ms)	17.119	13.212	110.128
Latência mediana (ms)	18.328	13.051	101.318
Maior latência observada (ms)	19.492	15.099	147.993
Taxa de timeout (%)	0	0	0
Taxa de erro técnico (%)	0	0	0

Análise: Kimi K2.7 (~13,2 s em média) e DeepSeek V4 Flash (~17,1 s, com dispersão possivelmente influenciada pela instabilidade de rede no período dos testes) ficam na mesma faixa, ambos adequados a processamento assíncrono de tickets. O Kimi K3 foi de 5 a 11× mais lento (81 a 148 s), com grande dispersão entre rodadas, em razão do volume variável de reasoning — inviável para uso em tempo real e limitado para triagem em alto volume. Com 3 rodadas, o p95 foi aproximado pela maior latência observada.

14. Comparação de consistência

Percentual de tickets com classificação idêntica (categoria, sentimento, urgência e risco de churn) em todas as rodadas válidas do modelo:

Métrica	DeepSeek V4 Flash	Kimi K2.7	Kimi K3
Consistência entre rodadas (%)	40	60	60
Tickets que variaram	1, 2 e 4	2 e 4	2 e 4
Formato idêntico entre rodadas	Não (fence em 2/3; envelope variou)	Não (fence em 1/3)	Sim (JSON puro em 3/3)

Análise: nenhum modelo foi totalmente consistente. Os tickets 2 (cobrança duplicada) e 4 (usabilidade ambígua) variaram em todos os modelos — são justamente os casos em que a calibragem de urgência e churn exige mais julgamento, o que reforça a necessidade de exemplos de calibragem no prompt e de revisão humana nesses perfis de ticket. O DeepSeek apresentou a menor consistência (40%), com variação adicional no ticket 1, e também variou o formato da saída (2 rodadas em bloco de código com array; 1 em JSON puro com envelope de objeto). O Kimi K3 teve consistência de classificação igual à do K2.7 (60%), mas formato perfeitamente estável — único modelo sem nenhum desvio de formato.

15. Adequação ao formato e ao tom

Métrica	DeepSeek V4 Flash	Kimi K2.7	Kimi K3
JSON válido no formato exigido (%)	33,3 (1/3)	66,7 (2/3)	100 (3/3)
Adequação de tom e formato (1 a 5)	4,0	4,0	5,0

Análise: os três modelos interpretaram corretamente o lote em todas as rodadas, respondendo os 5 tickets com todos os campos e valores permitidos. O DeepSeek V4 Flash envolveu a saída em bloco de código markdown em 2 de 3 rodadas; na rodada conforme, usou envelope de objeto por ticket em vez de array — interpretação razoável do lote, mas diferente das demais, o que exige normalização na integração. O desvio é contornável com parser tolerante a fences e a envelopes alternativos, ou com exemplos de saída no prompt. O Kimi K2.7 cometeu o desvio de bloco de código em 1 de 3 rodadas. O Kimi K3 retornou JSON puro em array nas 3 rodadas — único com conformidade integral e estável de formato.

16. Risco percebido de erro

Métrica	DeepSeek V4 Flash	Kimi K2.7	Kimi K3
Risco percebido (1 a 5)	2	2	2
Erros críticos (%)	0	0	0
Seguro para automação (%)	100	100	100

Análise: nenhum modelo cometeu erros críticos em nenhuma rodada válida (todos os tickets analisados, sem omissões e sem alucinações de fatos). Os três ficam com risco 2 (baixo), cada qual por um motivo: o DeepSeek pelos desvios de formato e pela menor consistência de classificação; o Kimi K2.7 pela superestimação recorrente de urgência e pelo fence em 1/3 das rodadas; o Kimi K3 pela superestimação de churn nas rodadas 2 e 3 e pela latência elevada e variável. Todos os falsos positivos observados (urgência ou churn acima do gabarito) desperdiçam esforço de triagem, mas não causam perda de tickets — o sentido mais seguro de errar.

17. Critérios eliminatórios

Valores utilizados: JSON válido $\geq 98\%$; erros críticos $\leq 5\%$; acurácia combinada de classificação $\geq 80\%$ (média das rodadas); p95 de latência dentro do limite aceitável para o produto (limite não definido pela equipe — critério não avaliado).

Critério	DeepSeek V4 Flash	Kimi K2.7	Kimi K3
JSON válido $\geq 98\%$	Reprovado (33,3%)*	Reprovado (66,7%)	Aprovado (100%)
Erros críticos $\leq 5\%$	Aprovado (0%)	Aprovado (0%)	Aprovado (0%)

Critério	DeepSeek V4 Flash	Kimi K2.7	Kimi K3
Acurácia combinada $\geq 80\%$	Aprovado (93,3%)	Reprovado (75%)	Aprovado (83,3%)
p95 dentro do limite	Não avaliado	Não avaliado	Não avaliado
Resultado nos critérios	Reprovado*	Reprovado	Aprovado

* O DeepSeek V4 Flash foi reprovado exclusivamente no critério de formato: o conteúdo JSON era válido e completo em todas as rodadas, mas 2 de 3 vieram em bloco de código markdown e a rodada conforme usou envelope de objeto em vez de array. É um desvio de apresentação mitigável (parser tolerante ou exemplos de saída no prompt), diferente de uma falha de qualidade — por isso a reprovação não inviabiliza o modelo, mas exige correção antes da adoção. O mesmo desvio, em menor grau, reprovou também o Kimi K2.7 no critério de formato.

18. Matriz de decisão

Pesos: qualidade 30%; risco de erro 20%; custo 20%; latência 15%; consistência 10%; adequação ao formato e tom 5% — aplicados integralmente. Notas normalizadas de 0 a 100 por critério (custo e latência normalizados em escala relativa entre os três modelos; risco invertido).

Critério	Peso	DeepSeek V4 Flash	Kimi K2.7	Kimi K3
Qualidade	30%	100	80	90
Risco de erro	20%	75	75	75
Custo	20%	100	83	0
Latência	15%	96	100	0
Consistência	10%	40	60	60
Adequação ao formato e tom	5%	80	80	100
Pontuação final (0 a 100)	—	87,4	80,7	53,0

19. Análise de trade-offs

- **Melhor qualidade:** DeepSeek V4 Flash (93,3% combinado médio, com rodadas de 100%, 85% e 95%), à frente do Kimi K3 (83,3%) e do Kimi K2.7 (75%). Amostra pequena, mas vantagem consistente nas 3 rodadas.
- **Menor custo:** DeepSeek V4 Flash no plano pago (US\$ 0,00022 por ticket — 12× mais barato que o K2.7 e 69× que o K3); camada gratuita disponível até 200 requisições/dia.
- **Menor latência:** Kimi K2.7 (~13,2 s em média), com o DeepSeek próximo (~17,1 s); Kimi K3 muito atrás (~110 s em média, com picos de 148 s).
- **Maior consistência:** Kimi K2.7 e Kimi K3 empatados (60%); DeepSeek atrás (40%). Nenhum modelo é plenamente consistente nos tickets 2 e 4.

- **Melhor formato:** Kimi K3 (JSON puro em 3/3, único sem desvios); Kimi K2.7 intermediário (2/3); DeepSeek com fence em 2/3 e envelope alternativo na rodada conforme — mitigável.
- **Menor risco:** empate entre os três (risco 2 — baixo), sem erros críticos em nenhuma rodada.
- **Critérios eliminatórios:** apenas o Kimi K3 foi integralmente aprovado. O DeepSeek foi reprovado apenas no formato (desvio mitigável); o Kimi K2.7 foi reprovado no formato (66,7%) e na acurácia combinada (75%).
- **Custo adicional justifica a qualidade?** Não: o Kimi K3, 69× mais caro que o DeepSeek pago, teve qualidade média inferior (83,3% vs. 93,3%), mesma consistência e latência muito pior — sua única vantagem é o formato perfeito, obtível no DeepSeek com tratamento de saída.
- **Impacto em escala:** o DeepSeek pago elimina o teto da camada gratuita a custo quase irrelevante (~US\$ 1,1 mil por milhão de execuções em lote), preservando a margem em qualquer volume projetado.
- **Automação completa:** os três modelos são seguros para automação com revisão humana dos casos sinalizados como urgência alta/crítica ou churn médio/alto; os tickets dos perfis 2 e 4 (cobrança e usabilidade ambígua) merecem atenção especial por terem variado em todos os modelos.
- **Estratégia híbrida:** os dados sustentam um modelo principal econômico com fallback, dispensando camada premium neste momento.

20. Recomendação final

Recomendação: DeepSeek V4 Flash (plano pago) como modelo principal, com correção do desvio de formato e melhoria de consistência via calibragem do prompt; Kimi K2.7 como fallback.

- **Modelo principal:** DeepSeek V4 Flash no plano pago — melhor pontuação ponderada (87,4), melhor qualidade média (93,3% combinado), custo de US\$ 0,00022 por ticket sem teto de uso e latência adequada a processamento assíncrono (~17 s). Condições para adoção: (a) mitigar os desvios de formato (bloco de código em 2/3 rodadas; envelope alternativo na rodada conforme) com parser tolerante ou exemplos de saída no prompt, validando a conformidade do JSON; (b) melhorar a consistência (40%) com exemplos de calibragem de urgência e churn nos tickets dos perfis 2 e 4; (c) confirmar qualidade e consistência em rodada ampliada; (d) para volumes muito baixos (até 200 requisições/dia), a camada gratuita pode ser usada sem custo.

- **Fallback:** Kimi K2.7 — baixo custo (US\$ 0,0027 por ticket) e a menor latência média; usar quando o principal estiver indisponível ou fora de conformidade. Ressalvas: reprovado em dois critérios eliminatórios (formato em 1/3 das rodadas e acurácia combinada de 75%), com superestimação recorrente de urgência — recomenda-se calibrar o prompt com exemplos antes do uso pleno.
- **Não recomendado como padrão neste momento:** Kimi K3 — único com formato perfeito e boa calibragem, mas qualidade média inferior à do DeepSeek, custo 69× maior e latência de 81–148 s, inviável para triagem em volume ou tempo real; pode ser reavaliado como camada premium para casos críticos se rodadas futuras demonstrarem vantagem concreta nesses casos.
- **Revisão humana:** obrigatória para tickets classificados como urgência alta/crítica ou risco de churn médio/alto, especialmente os perfis que variaram em todos os modelos (cobrança e usabilidade com sinal de churn).
- **Observação:** dada a amostra pequena (5 tickets, 3 execuções em lote por modelo), a decisão deve ser confirmada com uma rodada ampliada (30 tickets, 3 repetições, execução individual por ticket) antes da implementação em produção.

21. Limitações

- **Desvio de execução:** os 5 tickets foram enviados em um único prompt por execução (lote), em vez de uma execução individual por ticket; o plano original previa execuções individuais. Isso pode ter afetado o comportamento dos modelos, inclusive a variação de envelope observada no DeepSeek (array vs. objeto por ticket).
- **Rodadas em planos distintos do DeepSeek:** das 3 rodadas válidas do DeepSeek, 1 foi na camada gratuita e 2 no plano pago; as métricas de custo consideram apenas as rodadas pagas.
- **Primeira execução do DeepSeek descartada:** houve suspeita de instabilidade de rede durante o primeiro teste; a execução foi refeita e não entrou nos cálculos. A latência do modelo pode ter sido afetada pela mesma instabilidade no período.
- **Amostra pequena:** 5 tickets e 3 execuções por modelo são suficientes para a atividade acadêmica, mas não para uma decisão de produção.
- **Consistência medida em lote:** a métrica reflete a estabilidade em execuções em lote; a consistência em execuções individuais pode ser diferente.
- **Parâmetros não controlados:** temperatura e limite máximo de resposta não foram configurados no OpenCode.
- **Uso de cache variável:** várias rodadas reutilizaram cache de sessão (9.600 e 9.088 tokens no DeepSeek; 8.448 no Kimi K3), reduzindo custo e possivelmente

afetando latência; o efeito nas respostas não é mensurável com os dados disponíveis.

- **Coluna "Usage" não documentada:** a plataforma exibe US\$ 60 na coluna Usage do DeepSeek V4 Flash, cujo significado (pacote, crédito ou compromisso mínimo) não foi confirmado neste teste.
- **Preços e limites vigentes na data do teste:** os preços por milhão de tokens e o limite de 200 requisições/dia da camada gratuita são os informados pela plataforma em 30/07/2026 e podem mudar ou ser descontinuados.
- **Critério de p95 sem limite definido:** a equipe não definiu o limite aceitável de latência antes da execução; com 3 rodadas, o p95 foi apenas aproximado pelo máximo observado.
- **Subjetividade do gabarito:** urgência é o campo mais dependente de julgamento; divergências entre modelos e gabarito (ex.: tickets 1 e 2 classificados como "alta" pelos três modelos em ao menos uma rodada vs. "media" do gabarito) podem ser razoáveis.
- **Avaliação humana não cega:** o avaliador conhecia a identidade dos modelos.

22. Conclusão

O benchmark comparou DeepSeek V4 Flash, Kimi K2.7 e Kimi K3 na tarefa de análise automatizada de tickets, com dados reais de 3 execuções por modelo na plataforma OpenCode. O DeepSeek V4 Flash (plano pago) consolidou a melhor relação custo-benefício: melhor qualidade média do benchmark (93,3% de acurácia combinada, com rodadas de 100%, 85% e 95%), custo de apenas US\$ 0,00022 por ticket sem teto de uso e latência adequada (~17 s). Seus pontos fracos são claros e tratáveis: consistência de classificação de 40% (a menor) e desvios de formato em 2 de 3 rodadas (bloco de código; na rodada conforme, envelope alternativo). O Kimi K2.7 confirmou boa latência e baixo custo, mas a superestimação recorrente de urgência o reprovou no critério de qualidade; o Kimi K3 confirmou o formato perfeito, porém com qualidade média inferior à do DeepSeek, custo 69× maior e latência de 81–148 s — nenhuma vantagem que justifique a diferença.

A recomendação é adotar o **DeepSeek V4 Flash (plano pago) como modelo principal**, condicionado à mitigação dos desvios de formato e à melhoria de consistência via calibragem do prompt, com o **Kimi K2.7 como fallback e revisão humana** dos casos de alto risco e dos perfis de ticket que variaram em todos os modelos — a ser confirmada por uma rodada ampliada de testes antes da implementação em produção.